

READING SHEET

State & Events — useState

Module 3 · Frontend: React & Next.js · Day 28 · Week 6

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	Revision + mini-project	2–3 hours

Tip How to use this sheet

This is your take-away for Day 28 — the day your components stop being drawings and start responding to people. It revises `useState`, event handling, controlled inputs and lifting state up, continuing the Addis Eats menu from Day 27. Read it beside your editor and type every example: state is one of those topics that only makes sense once you have watched a component re-render in front of you.

Part 1 · State & useState

Props come from outside a component and can never be changed by it. **State** is different: it is a value the component owns, can change, and remembers between renders. Changing state also tells React to render the component again — which is what actually updates the screen.

Why a plain variable is not enough

A normal variable inside a component is created fresh on every render and thrown away when the function ends. Changing it does two things wrong: the new value is forgotten, and React is never told that anything needs redrawing.

```
function OrderCounter() {
  let count = 0;           // recreated on every render
  function add() {
    count = count + 1;     // changes, then is forgotten
  }
  return <button onClick={add}>Orders: {count}</button>;
}
```

this counter never updates

Declaring state

`useState` returns an array of exactly two items, which you destructure: the current value and a function that sets a new one. The argument you pass is the initial value, used on the first render only.

OrderCounter.jsx — useState

```
import { useState } from "react";

function OrderCounter() {
  const [count, setCount] = useState(0);
  //   value   updater   initial value

  return (
    <button onClick={() => setCount(count + 1)}>
      Orders today: {count}
    </button>
  );
}
```

Tip Naming and placement

Always name the pair `x / setX` — `count/setCount`, `category/setCategory`. And call `useState` at the top level of the component only: never inside an `if`, a loop or a nested function. React matches state to hooks by call order, so that order must be identical on every render.

State versus props

PROPS	STATE
Passed in by the parent	Owned by the component itself
Read-only inside the component	Changed by calling the setter
Change when the parent re-renders	Changes when you call the setter
Configure a component	Remember something over time
A dish's name and price	How many are in the cart

The test is simple: does this value ever change because of something happening *inside* this component? If yes, it is state. If it just arrives from outside, it is a prop.

Part 2 · Handling Events

Events are wired straight onto JSX elements as props. The prop name is camelCase and its value is a real JavaScript function in braces — not a string as it was in HTML.

OrderButton.jsx — onClick

```
function OrderButton() {
  function handleClick() {
    alert("Doro Wat added to your order");
  }
  return <button onClick={handleClick}>Order</button>;
}
```

Warning Pass the function, do not call it

`onClick={handleClick}` hands React a function to call later. `onClick={handleClick()}` calls it immediately during render and passes the result — usually undefined. If you need to pass an argument, wrap it: `onClick={() => addToOrder(dish.price)}`.

The events you will use most

EVENT PROP	FIRES WHEN
onClick	The element is clicked or tapped
onChange	An input, select or textarea value changes
onSubmit	A form is submitted
onKeyDown	A key is pressed while the element has focus
onMouseEnter	The pointer moves onto the element
onFocus / onBlur	The element gains or loses focus

One handler, many items

Inside a map, the arrow wrapper is created fresh for each item and closes over that item's data — so a single handler can serve the whole menu.

```
Menu.jsx – a running total in ETB

const [total, setTotal] = useState(0);

function addToOrder(price) {
  setTotal(total + price);
}

dishes.map(dish => (
  <button key={dish.id} onClick={() => addToOrder(dish.price)}>
    {dish.name} – {dish.price} ETB
  </button>
))
```

The event object

Every handler receives the event as its first argument. `event.target` is the element that fired it, so `event.target.value` gives the current text of an input. `event.preventDefault()` stops the browser's built-in behaviour — essential in a form's `onSubmit`, where the default is a full page reload that would wipe your state.

Part 3 · Controlled Inputs

A **controlled input** takes its value from state and reports every keystroke back through `onChange`. State becomes the single source of truth for the form: whatever is in state is what is on screen.

```
CustomerName.jsx

function CustomerName() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        value={name}
        onChange={e => setName(e.target.value)}
        placeholder="Your name" />
      <p>Delivering to: {name || "-"}</p>
    </div>
  );
}
```

Warning value and onChange travel together

Supplying value without onChange gives a read-only field and a console warning. Starting from useState() instead of useState("") makes the input uncontrolled on the first render, and React warns when it switches.

Several fields, one state object

Give each input a name attribute and use a computed key to update just that field. The spread copies the rest of the object so the other fields survive.

```
OrderForm.jsx – one handler for the whole form

const [form, setForm] = useState({
  name: "", phone: "", area: "Bole"
});

function handleChange(e) {
  const { name, value } = e.target;
  setForm({ ...form, [name]: value }); // copy, then override
}


```

Warning Never mutate state

form.phone = value does not re-render, and it breaks the comparison React uses to detect change. Always build a new object or array and pass that to the setter.

Validating as the user types

Because the value is in state you can check it on every render. Anything you can work out from state — like whether a TeleBirr number is valid — should be *derived*, not stored, so it can never go stale.

```
TeleBirr phone – live validation

const [phone, setPhone] = useState("");
const valid = /^(?:\+251|0)9\d{8}$/.test(phone);


```

Part 4 · Lifting State Up

State is private to the component that declares it, and data in React only flows *down*. So when two sibling components need the same value — a category bar that knows the choice and a dish list that needs it — neither of them can own it. Move the state to their closest common parent. That is lifting state up.

Menu.jsx – the parent owns the state

```
function Menu({ dishes }) {
  const [category, setCategory] = useState("All");

  const shown = category === "All"
    ? dishes
    : dishes.filter(d => d.category === category);

  return (
    <>
      <CategoryBar selected={category} onSelect={setCategory} />
      <DishList dishes={shown} />
    </>
  );
}
```

The parent passes the **value** down to whoever displays it and the **setter** down to whoever changes it. Both children stay stateless and simple.

CategoryBar.jsx – calls the parent's setter

```
function CategoryBar({ selected, onSelect }) {
  const cats = ["All", "Main", "Vegan", "Grill"];

  return cats.map(cat => (
    <button key={cat}
      className={cat === selected ? "chip on" : "chip"}
      onClick={() => onSelect(cat)}>
      {cat}
    </button>
  ));
}
```

Tip Where state should live

Put state in the lowest component that contains everyone who needs it. Too low and siblings cannot reach it; too high and unrelated parts of the application re-render for nothing. Note that `shown` is derived from `dishes` and `category` on every render — it is not state, and storing it would only be another value to keep in sync.

Assigned Reading

Required

1. [react.dev — State: a component's memory](#) What / why: the official introduction to `useState`, why a plain variable is not enough, and how re-rendering works.
2. [react.dev — Responding to events](#) What / why: event handlers, passing arguments, and the rules that catch out every beginner.

Optional — go deeper

- [Sharing state between components](#) — lifting state up, step by step, with a worked example.
- [Updating objects in state](#) — why state is treated as read-only, and how to copy it correctly.
- [Reacting to input with state](#) — thinking in state rather than in DOM manipulation: the mental shift.

Exercises

Work these in order and commit as you go. Each one adds a piece of interactivity to the menu you built yesterday.

1. Add a count state to Dish with an "Add" button that increases it, and show the count beside the dish name.
2. Give Menu a category state and render the chips from an array, highlighting the selected one.
3. Lift the category state into Menu, passing selected and onSelect down to CategoryBar.
4. Filter the dish list from the category state and keep the empty-state message from Day 27.
5. Keep a running order total in state and display it in ETB below the menu.
6. Build a controlled delivery form (name, phone, area) using a single state object and one change handler.
7. Validate the TeleBirr number as it is typed, and disable the submit button until it is valid.

Mini-Project · The Interactive Addis Eats Menu

Tip What you build

Turn yesterday's static menu into a working ordering screen: clickable category chips that filter the dishes, an order total that grows in ETB as dishes are added, and a validated TeleBirr delivery form. Every piece of interactivity comes from state — no DOM manipulation anywhere.

Requirements

- A Menu component that owns the category state and derives the filtered list from it.
- A stateless CategoryBar receiving selected and onSelect as props, with the active chip styled differently.
- A DishList that renders the filtered dishes with stable id keys and an empty state.
- An order total held in state, updated from a click handler and displayed in ETB.
- A controlled OrderForm with name, phone and area in one state object.
- Live validation of the TeleBirr number, with the submit button disabled until it passes.
- No console warnings, and everything pushed to GitHub.

A worked start

Get one piece of state driving one visible change first, then build outwards from it:

starter – a click-driven category filter

```
import { useState } from "react";
import { dishes } from "../data";

export default function Menu() {
  const [category, setCategory] = useState("All");
  const cats = ["All", "Main", "Vegan", "Grill"];

  const shown = category === "All"
    ? dishes
    : dishes.filter(d => d.category === category);

  return (
    <div>
      {cats.map(c => (
        <button key={c} onClick={() => setCategory(c)}>{c}</button>
      ))}
      {shown.length === 0
        ? <p>No dishes in this category yet.</p>
        : shown.map(d => <Dish key={d.id} {...d} />)}
    </div>
  );
}
```

What to submit

Your updated GitHub repository containing `src/Menu.jsx`, `src/CategoryBar.jsx`, `src/DishList.jsx`, `src/OrderForm.jsx` and `src/data.js`, with a commit message describing what became interactive.

Check yourself

- Does clicking a category chip change what the list shows, without reloading the page?
- Is the selected chip visually different from the others, driven by the same state?
- Does the order total increase by the right number of ETB when a dish is added?
- Is every form field controlled — value from state, `onChange` back into state?
- Did you copy the form object with the spread rather than assigning to a property?
- Does the TeleBirrr validation accept 0911223344 and +251911223344, and reject the rest?
- Does the state live in the lowest component that contains everyone who needs it?